



JAVA STUDY MATERIAL

UNIT I, II, III, IV



MAY 22, 2026
ASHWITH FRANK
ALPHAOMEGA STUDIO

UNIT I

2 Marks Questions

1. What is Object Oriented Programming (OOP)?

Object Oriented Programming (OOP) is a programming method where programs are created using **objects and classes**. It helps in organizing data and functions together.

Example:

A **Car** is an object.

Its:

- Data → color, speed
 - Functions → start(), stop()
-

2. Define the terms of OOP (All Concepts)

i) Class: A class is a blueprint for creating objects.

Example:

```
class Student
```

```
{  
    int id;  
}
```

ii) Object: An object is a real instance of a class.

Example:

```
Student s1 = new Student();
```

iii) Encapsulation: Wrapping data and methods together into one unit is called encapsulation.

Example:

```
class Demo
{
    private int x;

    void setX(int a)
    {
        x = a;
    }
}
```

iv) Data Abstraction: Showing only important details and hiding unnecessary details.

Example:

ATM machine shows options but hides internal process.

v) Inheritance: One class acquires properties of another class.

Example:

```
class Animal { }
class Dog extends Animal { }
```

vi) Polymorphism: One method behaves differently in different situations.

Example:

```
sum(int a, int b)
sum(float a, float b)
```

vii) Dynamic Binding: Method call is decided during runtime.

Example: Method overriding in inheritance.

viii) Message Passing: Objects communicate with each other using methods.

Example:

`obj.display();`

3. Mention any four advantages of OOP

1. Code reusability using inheritance
 2. Data security using encapsulation
 3. Easy to maintain programs
 4. Reduces code duplication
-

4. Mention any four application areas of OOP

1. Banking systems
 2. Game development
 3. Real-time systems
 4. Web applications
-

5. List any four features of Java

1. Platform independent
2. Object Oriented
3. Secure
4. Portable

6. What is Java Bytecode?

Java Bytecode is the intermediate code generated after compiling a Java program.

It is executed by the JVM.

File extension:

.class

7. What is Java Virtual Machine (JVM)?

JVM is a virtual machine that executes Java bytecode.

Functions of JVM:

- Runs Java programs
 - Converts bytecode into machine code
 - Makes Java platform independent
-

8. List different arithmetic operators available in Java

Operator Meaning

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
++	Increment
--	Decrement

9. What is type conversion? What are its types?

Type conversion means converting one data type into another data type.

Types:

1. Implicit Type Conversion
 2. Explicit Type Conversion
-

10. What is stream? List different types of streams in Java

A stream is a flow of data used for input and output operations.

Types of Streams:

1. Byte Stream
 2. Character Stream
-

5 Marks Questions

1. Explain various features of OOP

I) Class: A class is a blueprint for objects.

Example:

```
class Car
{
    String color;
}
```

ii) Object: An object is an instance of a class.

Example:

```
Car c1 = new Car();
```

iii) Encapsulation: Binding data and methods together into a single unit.

Advantages:

- Data security
- Prevents unauthorized access

Example:

```
class Student
{
    private int marks;

    void setMarks(int m)
    {
        marks = m;
    }
}
```

iv) Abstraction: Hiding implementation details and showing only necessary information.

Example: ATM machine.

v) Inheritance: One class inherits properties from another class.

Syntax:

```
class A { }
class B extends A { }
```

Advantage:

Code reusability.

vi) Polymorphism

One method performs different tasks.

Types:

- Method Overloading
- Method Overriding

Example:

```
add(int a, int b)
```

```
add(float a, float b)
```

vii) Dynamic Binding: Method call is decided during runtime.

viii) Message Passing: Objects communicate through methods.

Example:

```
obj.display();
```

2. What are identifiers? Write the rules to be followed when creating an identifier.

Identifier: Identifiers are names given to:

- Variables
- Methods
- Classes
- Objects

Example:

```
int age;
```

Here, age is an identifier.

Rules for Identifiers

1. Identifier can contain:
 - Letters
 - Digits
 - Underscore (`_`)
 - Dollar symbol (`$`)
2. Identifier should not start with a digit.

Correct: num1

Wrong: 1num

3. Spaces are not allowed.

Wrong: student name

4. Java keywords cannot be used.

Wrong: int class;

5. Identifiers are case-sensitive.

Age ≠ age

3. List and explain different primitive data types available in Java

Java has 8 primitive data types.

Data Type	Size	Example
-----------	------	---------

byte	1 byte	byte a = 10;
------	--------	--------------

short	2 bytes	short b = 100;
-------	---------	----------------

Data Type	Size	Example
-----------	------	---------

int	4 bytes	int c = 1000;
long	8 bytes	long d = 20000L;
float	4 bytes	float e = 10.5f;
double	8 bytes	double f = 99.99;
char	2 bytes	char g = 'A';
boolean	1 bit	boolean h = true;

Example Program

```
class DataTypes
{
    public static void main(String args[])
    {
        int a = 10;
        float b = 5.5f;
        char c = 'A';

        System.out.println(a);
        System.out.println(b);
        System.out.println(c);
    }
}
```

4. What are streams? List and explain different byte and character streams

Stream: A stream is a flow of data between source and destination.

Streams are used for input and output operations.

Types of Streams

1. Byte Streams: Used for binary data like:

- Images
- Audio
- Videos

Classes:

- FileInputStream
- FileOutputStream

Example:

```
FileInputStream fin;
```

```
FileOutputStream fout;
```

2. Character Streams: Used for text data.

Classes:

- FileReader
- FileWriter

Example:

```
FileReader fr;
```

```
FileWriter fw;
```

Difference Between Byte and Character Streams

Byte Stream	Character Stream
-------------	------------------

Handles binary data	Handles text data
---------------------	-------------------

Uses bytes	Uses characters
------------	-----------------

Faster for binary files	Better for text files
-------------------------	-----------------------

5. Explain the relational and logical operators with example

Relational Operators: Relational operators compare two values.

Operator Meaning

== Equal to

!= Not equal to

> Greater than

< Less than

>= Greater than or equal

<= Less than or equal

Example

```
class Relational
{
    public static void main(String args[])
    {
        int a = 10, b = 20;

        System.out.println(a < b);
    }
}
```

```
}  
}
```

Output:

true

Logical Operators

Logical operators combine conditions.

Operator Meaning

&& AND

! NOT

Example

```
class Logical  
{  
    public static void main(String args[])  
    {  
        int a = 10, b = 20;  
  
        System.out.println(a < b && b > 5);  
    }  
}
```

Output:

true

6. Explain different forms of if statement with syntax and example

1. Simple if Statement

Syntax

```
if(condition)
{
    statements;
}
```

Example

```
int a = 10;

if(a > 5)
{
    System.out.println("Greater");
}
```

2. if-else Statement

Syntax

```
if(condition)
{
    statements;
}
else
{
    statements;
}
```

Example

```
int a = 5;

if(a % 2 == 0)
{
    System.out.println("Even");
}
else
{
    System.out.println("Odd");
}
```

3. else-if Ladder

Syntax

```
if(condition1)
{
}
else if(condition2)
{
}
else
{
}
```

Example

```
int marks = 85;
```

```
if(marks >= 90)
{
    System.out.println("A");
}
else if(marks >= 75)
{
    System.out.println("B");
}
else
{
    System.out.println("C");
}
```

4. Nested if

Example

```
int a = 10;
```

```
if(a > 0)
{
    if(a < 20)
    {
        System.out.println("Valid");
    }
}
```


7. Explain switch statement with syntax and example

Switch Statement: Switch statement is used to select one block from many blocks.

Syntax

```
switch(expression)
{
    case value1:
        statements;
        break;
    case value2:
        statements;
        break;
    default:
        statements;
}
```

Example Program

```
class SwitchDemo
{
    public static void main(String args[])
    {
        int day = 2;
        switch(day)
        {
```

```
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    default:
        System.out.println("Invalid");
    }
}
}
```

Output

Tuesday

UNIT II

2 Marks Questions

1. What are arrays?

An array is a collection of similar data types stored in contiguous memory locations.

Example:

```
int marks[] = {10,20,30};
```

Here, multiple integer values are stored in a single array.

2. How to create and instantiate a two dimensional array? Give example.

A two-dimensional array stores data in rows and columns.

Syntax

```
datatype arrayname[][] = new datatype[row][column];
```

Example

```
int a[][] = new int[2][3];
```

This creates:

- 2 rows
 - 3 columns
-

3. List any four methods associated with Strings.

Method	Purpose
length()	Finds string length
charAt()	Returns character at position
equals()	Compares strings

Method	Purpose
toUpperCase()	Converts to uppercase

4. What is vector? How is it different from array?

Vector: A vector is a dynamic array that can grow or shrink automatically.

Difference Between Vector and Array

Array	Vector
Fixed size	Dynamic size
Faster	Slightly slower
Cannot grow automatically	Can grow automatically

5. What are classes and objects?

Class: A class is a blueprint for creating objects.

Example:

```
class Student
{
    int id;
}
```

Object: An object is an instance of a class.

Example:

```
Student s1 = new Student();
```

6. What are constructors? What is the purpose of a constructor?

A constructor is a special method used to initialize objects.

Characteristics:

- Constructor name should be same as class name.
- It does not have a return type.

Purpose:

- Initializes object data automatically.

Example:

```
class Demo
{
    Demo()
    {
        System.out.println("Constructor Called");
    }
}
```

7. What is the use of this keyword in Java?

The this keyword refers to the current object of the class.

Uses:

- Distinguishes instance variables from local variables.
- Calls current class methods or constructors.

Example:

```
class Test
{
    int x;
```

```
Test(int x)
{
    this.x = x;
}
}
```

5 Marks Questions

1. Explain for loop with syntax and example

for Loop: A for loop is used to repeat a block of code multiple times.

Syntax

```
for(initialization; condition; increment/decrement)
{
    statements;
}
```

Example Program

```
class ForDemo
{
    public static void main(String args[])
    {
        for(int i=1; i<=5; i++)
        {
            System.out.println(i);
        }
    }
}
```

```
}  
}
```

Output

```
1  
2  
3  
4  
5
```

Explanation

- `int i=1` → initialization
- `i<=5` → condition
- `i++` → increment

The loop runs until the condition becomes false.

2. Explain while loop with syntax and example

while Loop

A while loop executes repeatedly while the condition is true.

Syntax

```
while(condition)  
{  
    statements;  
}
```

Example

```
class WhileDemo
{
    public static void main(String args[])
    {
        int i = 1;

        while(i <= 5)
        {
            System.out.println(i);
            i++;
        }
    }
}
```

Output

```
1
2
3
4
5
```

3. Explain do while loop with syntax and example

do while Loop: The do while loop executes the block at least one time.

Syntax

```
do
{
    statements;
}
while(condition);
```

Example

```
class DoWhileDemo
{
    public static void main(String args[])
    {
        int i = 1;

        do
        {
            System.out.println(i);
            i++;
        }
        while(i <= 5);
    }
}
```

Output

```
1
2
```

3

4

5

4. Explain break and continue statement with suitable example

break Statement: The break statement terminates the loop immediately.

Example

```
for(int i=1; i<=5; i++)  
{  
    if(i==3)  
        break;  
  
    System.out.println(i);  
}
```

Output

1

2

continue Statement: The continue statement skips the current iteration.

Example

```
for(int i=1; i<=5; i++)  
{  
    if(i==3)  
        continue;  
  
    System.out.println(i);  
}
```

```
}
```

Output

1

2

4

5

5. What are arrays? Explain how to declare instantiate initialize and use a one-dimensional array with suitable code example.

Array: An array stores multiple values of the same type.

Declaration

Syntax

```
datatype arrayname[];
```

Example

```
int marks[];
```

Instantiation: Memory allocation for array.

Syntax

```
arrayname = new datatype[size];
```

Example

```
marks = new int[5];
```

Initialization: Assigning values to array elements.

Example

```
marks[0] = 10;
```

```
marks[1] = 20;
```

Complete Program

```
class ArrayDemo
{
    public static void main(String args[])
    {
        int marks[] = {10,20,30,40,50};

        for(int i=0; i<marks.length; i++)
        {
            System.out.println(marks[i]);
        }
    }
}
```

Output

10

20

30

40

50

6. Explain any four string methods with suitable example

1. length()

Returns length of the string.

Example

```
String s = "Java";  
System.out.println(s.length());
```

Output

4

2. charAt()

Returns character at given index.

Example

```
String s = "Java";  
System.out.println(s.charAt(1));
```

Output

a

3. toUpperCase()

Converts string to uppercase.

Example

```
String s = "java";  
System.out.println(s.toUpperCase());
```

Output

JAVA

4. equals()

Compares two strings.

Example

```
String a = "Java";  
String b = "Java";  
System.out.println(a.equals(b));
```

Output

true

7. Explain any five vector methods with suitable code example

Vector

Vector is a dynamic array in Java.

Common Vector Methods

Method	Purpose
add()	Adds element
remove()	Removes element
size()	Returns size
get()	Access element
clear()	Removes all elements

Example Program

```
import java.util.*;  
class VectorDemo  
{
```

```
public static void main(String args[])
{
    Vector v = new Vector();
    v.add("Java");
    v.add("Python");
    System.out.println(v);
    v.remove("Python");
    System.out.println(v.size());
    System.out.println(v.get(0));
    v.clear();
    System.out.println(v);
}
}
```

8. Write the general form of a class? Explain how to define a class in Java with suitable example.

General Form of a Class

Syntax

```
class ClassName
{
    variables;
    methods;
}
```

Example Program

```
class Student
```

```
{  
    int id;  
    String name;  
    void display()  
    {  
        System.out.println(id);  
        System.out.println(name);  
    }  
    public static void main(String args[])  
    {  
        Student s1 = new Student();  
        s1.id = 101;  
        s1.name = "Ash";  
        s1.display();  
    }  
}
```

Output

101

Ashwith

9. Explain how to create an object in Java with suitable example

Object Creation: An object is created using the new keyword.

Syntax

ClassName objectname = new ClassName();

Example Program

```
class Student
{
    int id;
    void show()
    {
        System.out.println("Object Created");
    }
    public static void main(String args[])
    {
        Student s1 = new Student();
        s1.id = 101;
        s1.show();
    }
}
```

Output

Object Created

10. Explain the use of Constructors with suitable example

Constructor: A constructor initializes object data automatically.

Characteristics

- Same name as class
- No return type
- Automatically called when object is created

Example Program

```
class Student
{
    int id;

    Student()
    {
        id = 100;
    }

    void display()
    {
        System.out.println(id);
    }

    public static void main(String args[])
    {
        Student s1 = new Student();
        s1.display();
    }
}
```

Output

100

11. Explain command line arguments in Java with example

Command Line Arguments: Command line arguments are values passed to the program during execution.

They are stored in: String args[]

Example Program

```
class CommandDemo
{
    public static void main(String args[])
    {
        System.out.println("Name: " + args[0]);
    }
}
```

Compilation

```
javac CommandDemo.java
```

Execution

```
java CommandDemo Ashwith
```

Output

```
Name: Ashwith
```

12. Explain method overloading with suitable example

Method Overloading

Method overloading means creating multiple methods with the same name but different parameters.

Example Program

```
class Add
{
```

```
int sum(int a, int b)
{
    return a+b;
}
int sum(int a, int b, int c)
{
    return a+b+c;
}
public static void main(String args[])
{
    Add obj = new Add();
    System.out.println(obj.sum(10,20));
    System.out.println(obj.sum(10,20,30));
}
}
```

Output

30

60

13. What is recursion? Illustrate recursion with a code example.

Recursion: Recursion is a process where a method calls itself repeatedly.

Example Program

```
class RecursionDemo
{
```

```
int fact(int n)
{
    if(n==1)
        return 1;

    return n * fact(n-1);
}

public static void main(String args[])
{
    RecursionDemo r = new RecursionDemo();

    System.out.println(r.fact(5));
}
}
```

Output

120

14. Explain static variable and methods with suitable example

Static Variable: A static variable is shared by all objects.

Static Method: A static method belongs to the class, not object.

Example Program

```
class Demo
{
```

```
static int count = 0;

static void show()
{
    System.out.println("Static Method");
}

public static void main(String args[])
{
    show();
    System.out.println(count);
}
}
```

Output

Static Method

0

15. Explain constructor overloading with suitable example

Constructor Overloading: Constructor overloading means using multiple constructors with different parameters in the same class.

Example Program

```
class Student
{
    int id;
    String name;
```

```
Student()
{
    id = 0;
    name = "Unknown";
}
Student(int i, String n)
{
    id = i;
    name = n;
}
void display()
{
    System.out.println(id + " " + name);
}
public static void main(String args[])
{
    Student s1 = new Student();
    Student s2 = new Student(101, "Ashwith");
    s1.display();
    s2.display();
}
}
```

Output

0 Unknown

101 Ashwith

UNIT – III

2 Marks Questions

1. What is inheritance?

Inheritance is a process where one class acquires properties and methods of another class.

Advantage:

- Code reusability

Example:

```
class A { }
```

```
class B extends A { }
```

2. What is the purpose of keyword extends?

The extends keyword is used to inherit one class from another class.

Syntax:

```
class Child extends Parent
```

```
{  
}
```

3. What is the purpose of super()?

super() is used to call the constructor of the parent class.

Example: super();

4. What are abstract classes?

An abstract class is a class declared using the abstract keyword. It cannot be instantiated.

Example:

```
abstract class Shape  
{  
}
```

5. What are abstract methods?

An abstract method is a method without a body.

Syntax: abstract void display();

6. How to prevent methods from overriding?

Use the final keyword to prevent method overriding.

Example:

```
final void show()  
{  
}
```

7. What are final variables?

A final variable is a constant variable whose value cannot be changed.

Example: final int x = 10;

8. What is the purpose of final class?

A final class prevents inheritance.

Example:

```
final class Demo  
{
```

```
}
```

9. What are final methods?

Final methods cannot be overridden by subclasses.

Example:

```
final void display()
```

```
{
```

```
}
```

10. How to define a package?

A package is defined using the package keyword.

Syntax: package mypack;

11. How to import a package?

A package is imported using the import keyword.

Syntax: import java.util.*;

12. List any four API packages of Java.

1. java.lang
 2. java.util
 3. java.io
 4. java.awt
-

13. What are different methods of creating threads?

Two methods:

1. Extending Thread class
 2. Implementing Runnable interface
-

14. List different states of a thread.

1. New
 2. Runnable
 3. Running
 4. Waiting
 5. Dead
-

15. List any four thread methods.

1. start()
 2. sleep()
 3. stop()
 4. join()
-

16. What is the purpose of exception handler?

An exception handler handles runtime errors and prevents program termination.

Advantage:

- Maintains normal program flow
-

17. List any four types of exceptions in Java.

1. ArithmeticException
2. NullPointerException

3. `ArrayIndexOutOfBoundsException`

4. `NumberFormatException`

18. What is the purpose of try and catch block?

- try block contains risky code.
- catch block handles exceptions.

Example:

```
try
{
    int a = 10/0;
}
catch(Exception e)
{
    System.out.println(e);
}
```

5 Marks Questions

1. Explain single inheritance with suitable example

Single Inheritance: When one child class inherits one parent class, it is called single inheritance.

Syntax

```
class Parent
{
}
```

```
class Child extends Parent
{
}
```

Example Program

```
class Animal
{
    void eat()
    {
        System.out.println("Eating");
    }
}

class Dog extends Animal
{
    void bark()
    {
        System.out.println("Barking");
    }
}

public static void main(String args[])
{
    Dog d = new Dog();

    d.eat();
}
```

```
        d.bark();  
    }  
}
```

Output

Eating

Barking

2. Explain method overriding with suitable example

Method Overriding: Method overriding occurs when child class provides its own implementation of parent class method.

Example Program

```
class Animal  
{  
    void sound()  
    {  
        System.out.println("Animal Sound");  
    }  
}  
  
class Dog extends Animal  
{  
    void sound()  
    {  
        System.out.println("Dog Barks");  
    }  
}
```

```
}  
  
public static void main(String args[])  
{  
    Dog d = new Dog();  
    d.sound();  
}  
}
```

Output

Dog Barks

3. Explain the purpose abstract and final classes

Abstract Class: An abstract class is declared using abstract keyword.

Features:

- Cannot create objects
 - Used for abstraction
 - May contain abstract methods
-

Example

```
abstract class Shape  
{  
    abstract void draw();  
}
```

Final Class: A final class cannot be inherited.

Purpose:

- Prevents inheritance
 - Provides security
-

Example

final class Demo

```
{  
}
```

Difference**Abstract Class**

Used for inheritance

Can contain abstract methods

Final Class

Prevents inheritance

Cannot be extended

4. Explain how to create and use a package in Java with suitable example

Package: A package is a collection of related classes.

Creating a Package

Syntax: package mypack;

Example**Step 1: Create Package Class**

package mypack;


```
public class Test
{
    public void show()
    {
        System.out.println("Package Example");
    }
}
```

Step 2: Use the Package

```
import mypack.Test;

class Demo
{
    public static void main(String args[])
    {
        Test t = new Test();
        t.show();
    }
}
```

Output

Package Example

5. List and explain different API packages available in Java

Java API Packages: API packages contain predefined classes and methods.

1. java.lang

Provides basic classes.

Examples:

- String
 - Math
 - System
-

2. java.util

Provides utility classes.

Examples:

- Scanner
 - Vector
 - Date
-

3. java.io

Used for input and output operations.

Examples:

- FileReader
 - FileWriter
-

4. java.awt

Used for GUI applications.

Examples:

- Button
- Frame

5. java.applet

Used for applet programming.

6. List and explain any five thread methods

Method Purpose

start() Starts thread

sleep() Pauses thread

stop() Stops thread

join() Waits for thread completion

isAlive() Checks thread status

Example Program

```
class MyThread extends Thread
{
    public void run()
    {
        System.out.println("Thread Running");
    }
    public static void main(String args[])
    {
        MyThread t = new MyThread();
        t.start();
        System.out.println(t.isAlive());
    }
}
```

}

7. Explain the life cycle of a Thread with diagram

Life Cycle of Thread: A thread passes through different states during execution.

States of Thread

1. New
 2. Runnable
 3. Running
 4. Waiting / Blocked
 5. Dead
-

Diagram

New → Runnable → Running → Dead

↓

Waiting

Explanation

1. New

Thread object is created.

2. Runnable

Thread is ready to run.

3. Running

Thread is executing.

4. Waiting

Thread waits temporarily.

5. Dead

Thread execution ends.

8. Explain exception handling in Java with suitable example

Exception Handling: Exception handling is used to handle runtime errors.

Keywords Used

- try
 - catch
 - finally
 - throw
 - throws
-

Example Program

```
class ExceptionDemo
{
    public static void main(String args[])
    {
        try
        {
            int a = 10/0;
            System.out.println(a);
        }
    }
}
```

```
        catch(ArithmeticException e)
        {
            System.out.println("Cannot divide by zero");
        }
        finally
        {
            System.out.println("Program Ended");
        }
    }
}
```

Output

Cannot divide by zero

Program Ended

9. List and explain any 5 built-in exceptions of Java

Exception	Meaning
ArithmeticException	Error in arithmetic operations
NullPointerException	Accessing null object
ArrayIndexOutOfBoundsException	Invalid array index
NumberFormatException	Invalid number conversion
ClassNotFoundException	Class not found

Example

```
int a = 10/0;
```

This causes:

ArithmeticException

UNIT – IV

2 Marks Questions

1. What is an Applet?

An applet is a small Java program that runs inside a web browser or appletviewer.

2. List two types of Applets available in Java.

1. Local Applet
 2. Remote Applet
-

3. What is the purpose of appletviewer?

appletviewer is a tool used to run and test applets without using a browser.

4. List any two features of Applets.

1. Platform independent
 2. Secure execution environment
-

5. List any four methods associated with Applets.

1. init()
 2. start()
 3. stop()
 4. destroy()
-

6. List any 4 methods defined by Applet.

1. init()

2. start()
 3. paint()
 4. destroy()
-

7. What is an event? Give example.

An event is an action performed by the user.

Examples:

- Mouse click
 - Key press
 - Button click
-

8. What is the purpose of Event Listener?

An Event Listener handles events generated by components.

Example:

ActionListener handles button click events.

9. List the key features of a swing GUI

1. Platform independent
 2. Lightweight components
 3. Rich GUI controls
 4. Customizable
-

10. List any four swing components.

1. JButton
2. JLabel

- 3. JTextField
- 4. JCheckBox

5 Marks Questions

1. Write the complete Applet skeleton with example

Applet Skeleton

Example Program

```
import java.applet.*;
import java.awt.*;
public class Simple extends Applet
{
    public void init()
    {
    }
    public void start()
    {
    }
    public void paint(Graphics g)
    {
        g.drawString("Welcome to Applet",50,50);
    }
    public void stop()
    {
    }
}
```

```
public void destroy()  
{  
}  
}
```

HTML File

```
<html>  
  
<body>  
  
<applet code="Simple.class" width="300" height="300">  
  
</applet>  
  
</body>  
</html>
```

2. List and explain any 6 methods defined by Applet

Method Purpose

init()	Initializes applet
start()	Starts applet
stop()	Stops applet
destroy()	Destroys applet
paint()	Displays output
repaint()	Calls paint() again

3. List and explain the components of Delegation Event Model

Delegation Event Model Components

1. Event Source: Component that generates event.

Example: Button

2. Event Object: Contains event information.

Example: ActionEvent

3. Event Listener: Handles the event.

Example: ActionListener

Flow: Source → Event Object → Listener

4. List and explain any three Event Listener Interfaces

Listener	Purpose
ActionListener	Handles button click
MouseListener	Handles mouse events
KeyListener	Handles keyboard events

Example `button.addActionListener(this);`

5. Explain how any two Mouse events are handled in Applets with suitable example

Mouse Events: Mouse events are handled using MouseListener.

Common Mouse Events

1. mouseClicked()
 2. mousePressed()
-

Example Program

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;

public class MouseDemo extends Applet implements MouseListener
{
    String msg="";

    public void init()
    {
        addMouseListener(this);
    }

    public void mouseClicked(MouseEvent me)
    {
        msg="Mouse Clicked";
        repaint();
    }

    public void mousePressed(MouseEvent me)
    {
        msg="Mouse Pressed";
        repaint();
    }
}
```

```
public void paint(Graphics g)
{
    g.drawString(msg,50,50);
}
public void mouseReleased(MouseEvent me){}
public void mouseEntered(MouseEvent me){}
public void mouseExited(MouseEvent me){}
}
```

6. Explain how any two Key events are handled in Applets with suitable example

Key Events: Key events are handled using KeyListener.

Common Key Events

1. keyPressed()
 2. keyReleased()
-

Example Program

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;
public class KeyDemo extends Applet implements KeyListener
{
    String msg="";
    public void init()
```

```
{  
    addKeyListener(this);  
}  
public void keyPressed(KeyEvent ke)  
{  
    msg="Key Pressed";  
    repaint();  
}  
public void keyReleased(KeyEvent ke)  
{  
    msg="Key Released";  
    repaint();  
}  
public void keyTyped(KeyEvent ke){}  
public void paint(Graphics g)  
{  
    g.drawString(msg,50,50);  
}  
}
```